

Projet d'étude — Mastère DevOps

Rendu individuel

Elyess Rjafellah — CI/CD & automatisation

Document technique final — partie individuelle

SUP DE VINCI — Promotion 2025-2026

Code promo : **CODEPROMO** (à compléter)

1. Contribution au projet

J'ai conçu et fiabilisé la chaîne d'intégration et de déploiement continu (GitHub Actions), afin que tout le cycle — provisionnement, build, scan, déploiement, sauvegarde — soit entièrement automatisé.

Réalisations principales

- Conception des workflows GitHub Actions : intégration (lint + tests), sécurité, déploiement complet et sauvegarde planifiée.
- Automatisation du provisionnement (Terraform) et création automatique de l'état distant.
- Mise en place d'un pipeline **auto-réparant** : libération des verrous d'état, nettoyage des releases Helm bloquées, ordre d'installation des dépendances (CRDs avant l'application).
- Optimisation des temps d'exécution (suppression des attentes bloquantes inutiles).

2. Perspectives d'évolution

Réflexion sur l'avenir de l'infrastructure / de la solution sur mon périmètre :

- Passage en GitOps (ArgoCD) pour un déploiement déclaratif piloté par l'état du dépôt.
- Authentification OIDC fédérée GitHub → Azure (suppression des secrets longue durée).
- Environnements multiples (dev/staging/prod) avec promotion contrôlée.
- Tests de bout en bout (E2E) automatisés post-déploiement dans le pipeline.

3. Analyse critique des limites techniques rencontrées

- L'enchaînement provision + déploiement dans un seul workflow est pratique mais long ; un découpage en jobs/étapes parallélisés serait plus efficace.
- La connexion Azure par identifiant/mot de passe impose un compte sans MFA — peu adapté à la production.
- L'absence d'environnement de staging fait que les tests se font directement sur l'unique cluster.
- Les déclencheurs sur push peuvent lancer des déploiements coûteux : un garde-fou serait souhaitable.

4. Annexes

4.1 Documentation utilisateur (extrait de mon périmètre)

- Déploiement : Actions → workflow Deploy → *deploy* ; destruction : action *destroy*.
- Sauvegarde/restauration : workflow Backup (modes *backup* / *restore*), planifié quotidiennement.
- Les secrets nécessaires (identifiants Azure) sont décrits dans la documentation d'installation.

4.2 Analyse personnelle

Défis rencontrés

Le défi majeur a été de rendre le pipeline robuste face aux états intermédiaires (run interrompu, release Helm en échec, verrou d'état Terraform résiduel). J'ai progressivement ajouté des mécanismes d'auto-réparation pour que la chaîne se rétablisse seule sans intervention manuelle.

Forces

- Rigueur dans l'enchaînement et l'idempotence des étapes.
- Capacité à diagnostiquer et corriger rapidement les échecs de pipeline.

Faiblesses / points de vigilance

- Workflows monolithiques à découper davantage.
- Sécurisation de l'authentification CI à approfondir (OIDC).

Compétences développées

- GitHub Actions (workflows, secrets, déclencheurs, concurrency).
- Automatisation Terraform et gestion d'état distant.
- Diagnostic et résilience des pipelines CI/CD.

Axes d'amélioration personnels

- Modulariser les pipelines (jobs réutilisables, environnements).
- Mettre en place une stratégie multi-environnements et le GitOps.